

FraudShield

Real-Time Credit Card Fraud Detection Gateway

Project Synopsis

Submitted by: Ayush Sharma

Problem Statement

Rule-based fraud detection — fixed thresholds on amount, velocity, or location — catches only what it was explicitly written to catch. Tuned loosely, it misses novel fraud patterns; tuned aggressively, it blocks large numbers of legitimate transactions and creates real customer friction. Most small and mid-sized platforms lack a system that combines the speed of simple rules with a trained model that generalizes beyond them, routes genuinely ambiguous cases to a human analyst instead of guessing, and treats the detection threshold as a deliberate, documented trade-off rather than a hardcoded default.

Proposed System

FraudShield is a full-stack fraud-detection gateway serving four roles — Customer, Merchant, Fraud Analyst, and Admin. Each simulated transaction passes through a rule-based pre-filter, then a trained classifier that returns a fraud-probability score; the combined result routes it to auto-approve, auto-decline, or a Fraud Analyst's review queue, where every decision is logged as labeled data for future retraining. Admins can adjust the decision threshold directly, with the underlying trade-off made explicit rather than hidden. The classifier is trained separately on two datasets serving different purposes — IEEE-CIS Fraud Detection (590,540 real-world e-commerce transactions) as the primary production model, and a large, Sparkov-simulated transactions dataset ($\approx 1.85\text{M}$ rows) with fully readable transaction-category, merchant, and location columns, kept specifically for SHAP-based explainability.

Objectives

- Train and evaluate a machine-learning fraud classifier on real-world-scale transaction data, using metrics suited to severe class imbalance — precision, recall, F1-score, and AUC-ROC — rather than raw accuracy.
- Combine a fast rule-based pre-filter with the trained model into a single real-time scoring pipeline.
- Build a role-based web application so Customers, Merchants, Fraud Analysts, and Admins each get the view and permissions appropriate to their role.
- Give Fraud Analysts a prioritized review queue, and Admins a way to adjust — and justify — the detection threshold.
- Track model performance over time as analyst-labeled outcomes accumulate.
- Deploy a working, end-to-end version of the system for live demonstration.

Scope

In scope: a simulated transaction gateway (not a live payment processor), the full rule-based + ML detection pipeline, all four role-based interfaces, threshold management, performance analytics, and analyst notifications. Model training draws on the IEEE-CIS dataset as the primary source and a Sparkov-simulated transactions dataset as a secondary, explainability-focused source, trained separately rather than merged since the two feature schemas are incompatible.

Out of scope for this build, left as future enhancements: real (sandboxed or live) payment-gateway integration, streaming architectures such as Kafka, deep-learning approaches such as autoencoders or graph neural networks, and continuous/online learning.

Technology Stack

- **Frontend:** React (Vite), Tailwind CSS
- **Backend:** Django, Django REST Framework
- **Authentication:** Django auth + djangorestframework-simplejwt (JWT), role-based permissions
- **ML / Data:** scikit-learn, pandas, NumPy, imbalanced-learn (SMOTE), joblib
- **Database:** PostgreSQL, via Neon (pooled connection) in production

- **Visualization:** Recharts
- **Notifications:** Twilio / Django email
- **Deployment:** Vercel (frontend + backend) + Neon (database)

Expected Outcomes

- A deployed web application that scores simulated transactions in real time and routes them to approve, decline, or analyst review.
- A trained classifier that measurably outperforms the rule-based pre-filter alone on held-out test data, reported via precision, recall, F1-score, and AUC-ROC.
- Exploratory and geographic visualizations of the data, plus confidence intervals around the reported metrics.
- A working Fraud Analyst workflow where review decisions are captured as labelled data for future retraining.
- An Admin-configurable detection threshold with a documented rationale for the value chosen.
- A performance-analytics view showing model metrics trending as labelled data accumulates.
- A live demo in which the trained model catches at least one fraud case a rule-only check would have missed.

Course Component Coverage

All five components are covered. Full Stack — a genuine client-server system with role-aware data behind authenticated APIs, a separate consuming frontend, and a real deployment. Software Engineering — enforced role-based access, modular separation of concerns, automated tests covering permissions and end-to-end behaviour, and a documented trade-off behind at least one operational decision. Machine Learning, the strongest component — a genuine, supervised classification problem on realistically imbalanced data, correct methodology around a known leakage pitfall, imbalance-appropriate evaluation, and two independently trained models serving two different purposes. Data Visualization — an exploratory pass over the raw data, a geographic view of where fraud concentrates, and an operational dashboard tracking performance over time. Statistics & Probability — descriptive characterization of imbalance and missing data, plus a confidence interval around the reported metrics, turning a single number into a defensible estimate.

Functional Requirements

- **Customer:** Register/Log in; submit a simulated transaction (merchant, amount, required fields); view the outcome (approved / declined / under review).
- **Merchant:** View own transactions and their status.
- **Fraud Analyst:** View a review queue of flagged transactions sorted by score; approve or decline each; receive alerts for urgent cases; view model performance analytics.
- **Admin:** Adjust the detection threshold; manage user accounts and roles; view system-wide analytics.
- **System:** Score every transaction through the rule pre-filter and trained classifier combined; log every analyst decision as labelled data for retraining; replay held-out test transactions against the live endpoint for validation.

Non-Functional Requirements

- **Performance:** Scoring returns fast enough for a real-time decision, accounting for cold-start latency after idle periods.
- **Security:** Authenticated, role-based authorization enforced server-side, not just hidden in the UI.
- **Reliability:** Serialized model and its training environment must match at serving time, or inference fails silently.
- **Usability:** Non-technical roles (Merchant, Admin) can read their dashboards without ML background.
- **Maintainability:** Business logic, auth, ML serving, and reporting stay in independent modules.
- **Auditability:** Every analyst decision and every threshold change is logged with a rationale, not just a value.

Stakeholders & User Stories

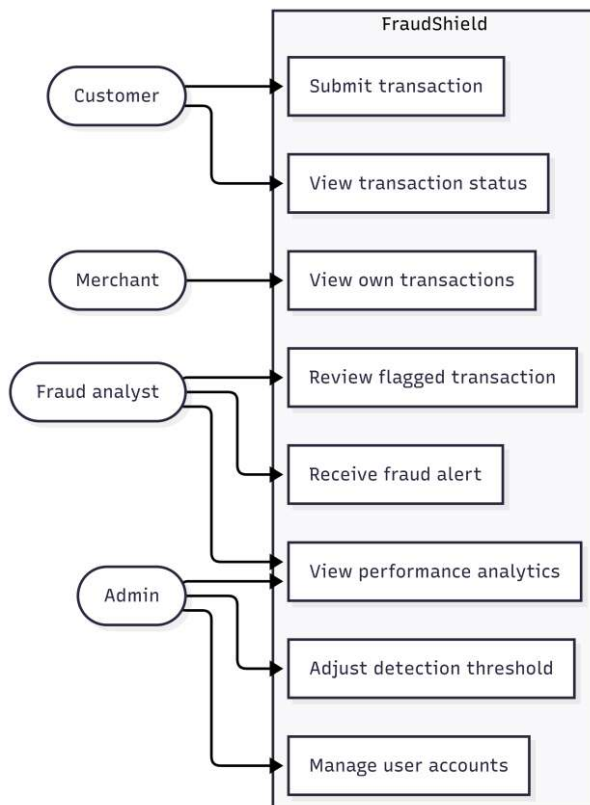
Customer: Wants a fast, fair decision

Merchant: Wants visibility into outcomes, minimal false declines

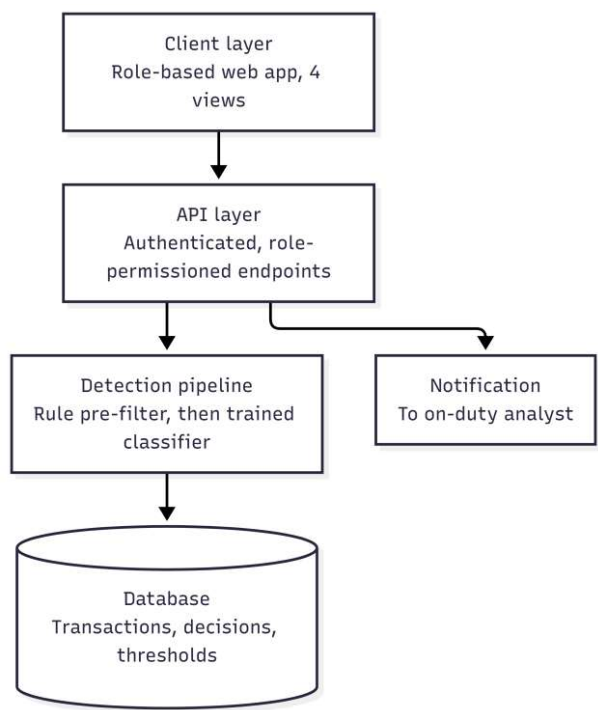
Fraud Analyst: Wants a manageable, trustworthy queue

Admin: Owns the threshold trade-off and user management

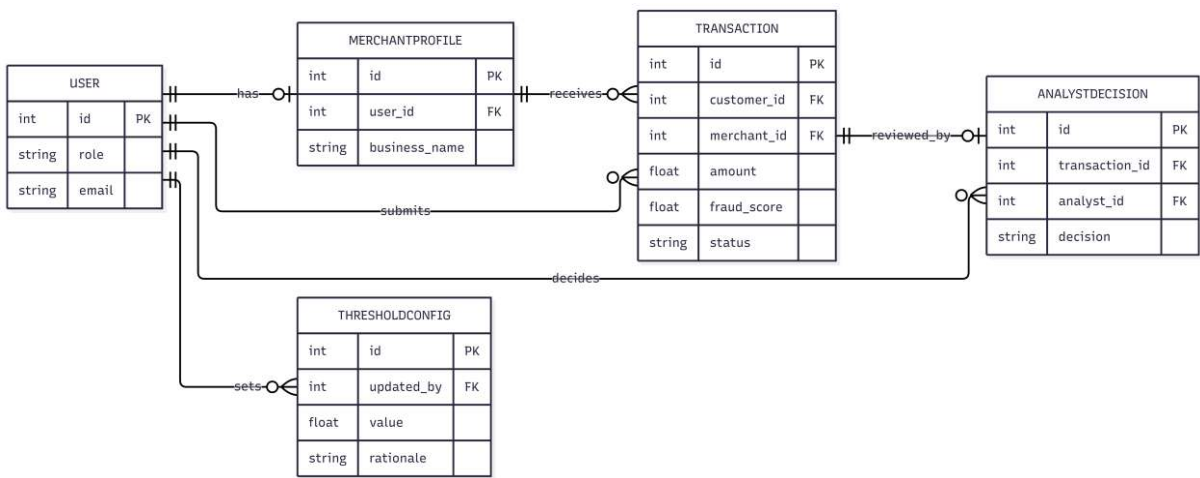
- As a Customer, I want to know immediately whether my transaction was approved, declined, or is under review, so I know what to expect next.
- As a Merchant, I want to see the status of transactions tied to my business, so I can reconcile my own records.
- As a Fraud Analyst, I want a prioritized queue of only the transactions that need me, so I'm not wasting time on obviously legitimate ones.
- As a Fraud Analyst, I want to see why a transaction was flagged, so I can decide quickly and with confidence.
- As an Admin, I want to adjust the threshold and understand its trade-off, so I can balance fraud losses against customer friction.



System Architecture



Database Design



Transaction scoring flow

